

cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith^{1,*}, Gregory Koytiger^{1,*}

¹Synthesize Bio

*Correspondence:

maxrhhighsmith@gmail.com; greg@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq analysis. Its statistical pipeline is rigorous but expensive to repeat at scale because it fits thousands of gene-level models. We asked whether those fits could run in parallel on a GPU without changing the analysis.

Results: We developed cuDESeq2, a GPU implementation of the DESeq2 workflow. It processes genes in batches and uses a fused kernel for the most costly dispersion calculations. Across six real-data designs, its intermediate estimates and final calls closely matched R DESeq2 1.52.0. Significant-gene sets had Jaccard similarity of at least 0.99963. On one A100, complete analyses were $3.6\text{--}35.1\times$ faster than 12-worker R. The same implementation also handled a 2,451-sample, six-tissue GTEx model. These results show that DESeq2-style inference can be accelerated without replacing it with a different statistical test.

Availability and implementation: Open source under the MIT license at https://github.com/synthesizebio/gpu_deseq; PyTorch and a CUDA GPU are required for the reported acceleration.

Contact: maxrhhighsmith@gmail.com; greg@synthesize.bio

Supplementary information: Appended to this manuscript.

1 Introduction

Differential-expression analysis turns RNA-seq counts into estimates of how gene expression changes between conditions. DESeq2 (Love et al., 2014) is a common choice for this task. It models count variability, shares information across genes, limits the influence of outliers, and corrects for multiple testing. It can also shrink noisy log fold changes (LFCs) with apeGLM (Zhu et al., 2019). Together, these steps produce stable effect estimates and calibrated significance calls.

Most of the work is repeated for every gene. Dispersion estimation, GLM fitting, and LFC shrinkage each solve thousands of small optimization problems. R DESeq2 can divide genes among CPU workers, but the calculations remain a good match for the much wider parallelism of a GPU. The challenge is fidelity. DESeq2 includes line searches, convergence tests, empirical-Bayes estimates, and fallbacks for unusual genes. A GPU port must preserve those decisions, not just the underlying model (Fig. 1).

This cost matters when the analysis is repeated. Examples include perturbation screens, resampling studies, many planned contrasts, and evaluation of gene-expression models. Recent virtual-cell benchmarks sometimes use cheaper rank-based tests in place of count models (Roohani et al., 2025; Wei et al., 2026). Faster DESeq2-style analysis would make that compromise less necessary. The need is growing alongside foundation models for gene expression (Theodoris et al., 2023; Cui et al., 2024; Bunne et al., 2024; Adduri et al., 2025; Koytiger et al., 2025).

Related tools solve different parts of the problem. glmGamPoi accelerates Gamma-Poisson GLMs on large sparse matrices (Ahlmann-Eltze and Huber, 2020). PyDESeq2 provides a Python implementation with similar, but not identical, results (Muzellec et al., 2023). GPU libraries have also sped up PCA, neighbor graphs, and clustering in single-cell workflows (Hu et al., 2025). None of these efforts establishes whether the DESeq2 workflow itself can move to a GPU while retaining close agreement with R.

We address that question with cuDESeq2. The implementation uses PyTorch (Paszke

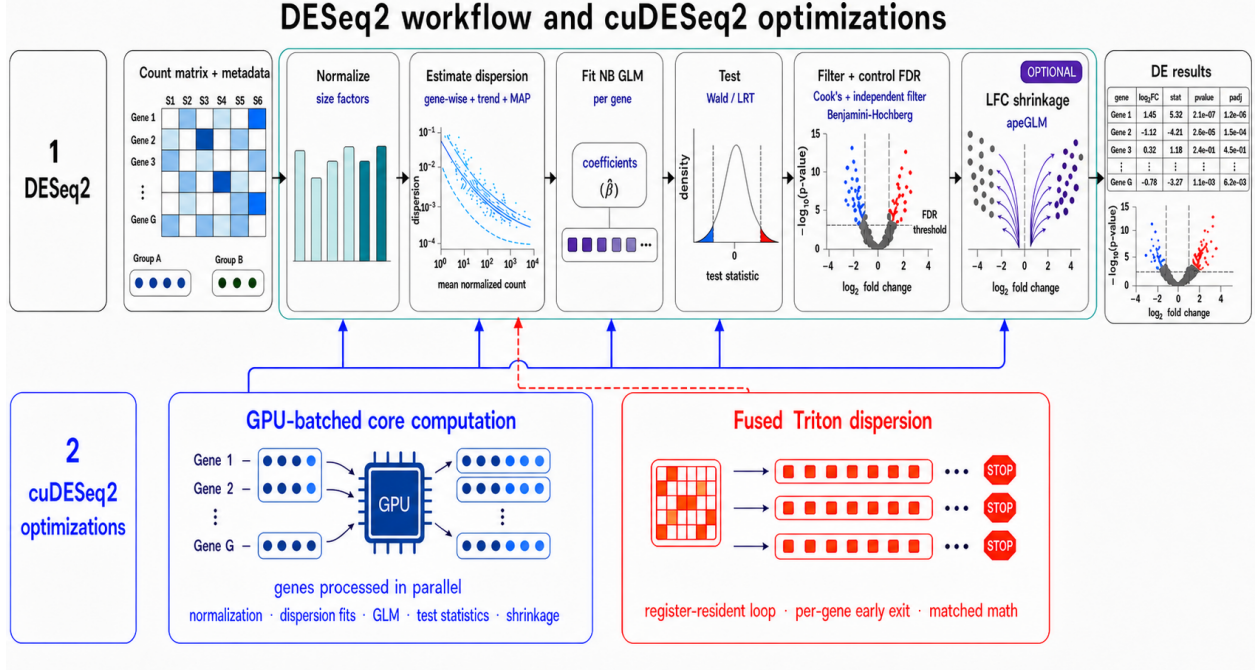


Figure 1: DESeq2 workflow and its GPU implementation. The upper path shows the statistical analysis. The lower path shows the two main acceleration ideas: process genes together on the GPU, and fuse the iterative dispersion fit into one Triton kernel. CUDA-graph replay provides a third dispersion mode.

et al., 2019) to batch gene-level work. It includes outlier replacement and apeGLM shrinkage. We also add CUDA-graph (NVIDIA Corporation, 2024) and Triton (Tillet et al., 2019) paths for dispersion estimation. We test agreement at each analysis stage, not only in the final gene list. We then measure end-to-end performance and GPU memory on published datasets and nested GTEx cohorts.

2 Methods

2.1 Reference workflow and supported scope

DESeq2 (Love et al., 2014) fits a negative-binomial GLM to each gene. For gene g , let K_{gi} be the count in sample i , s_i the sample's size factor, and $X \in \mathbb{R}^{S \times P}$ the design matrix. The model is $K_{gi} \sim \text{NB}(\mu_{gi}, \alpha_g)$, where $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and the variance is $\mu_{gi} + \alpha_g \mu_{gi}^2$.

The workflow has five main stages. It normalizes for library depth, estimates and moderates dispersion, fits the GLM, tests coefficients, and adjusts for multiple testing. Optional apeGLM shrinkage stabilizes noisy LFC estimates but does not change the test p-values. cuDESeq2 preserves this order and passes the same quantities from one stage to
60 the next.

2.1.1 Normalization

Median-of-ratios normalization uses GPU reductions and quantiles. As in DESeq2, `sfType="ratio"` raises an error when every gene contains a zero. The `sfType="poscounts"` alternative supports such sparse matrices.

65 2.1.2 Dispersion estimation

Both gene-wise dispersion fits—MLE and MAP—run as batches. Their execution modes are described in Sec. 2.2.

2.1.3 GLM fitting and hypothesis testing

Negative-binomial IRLS and the Wald or likelihood-ratio tests are also batched over genes
70 (Sec. 2.3).

2.1.4 Significance filtering and result assembly

Cook’s filtering, independent filtering, and multiple-testing adjustment use vectorized operations. In a CUDA run, Cook’s distances and count replacement remain on the GPU. Independent filtering evaluates 50 nested thresholds from a single p-value ordering. It
75 performs the full Benjamini–Hochberg adjustment only for the selected threshold. Eligible outliers are then replaced and the affected genes refitted, as in DESeq2.

2.1.5 LFC shrinkage

The apeGLM optimizer runs as a gene batch on the GPU (Sec. 2.4).

2.2 Dispersion acceleration

80 The dispersion stage follows DESeq2’s `fitDisp` and `estimateDispersions` logic. For each gene, it maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\text{CR}}(\alpha_g) = \sum_{i=1}^S \log \text{NB}(K_{gi}; \mu_{gi}, \alpha_g) - \frac{1}{2} \log \det(X^\top W_g X), \quad (1)$$

where $W_g = \text{diag}(\mu_{gi}/(1 + \alpha_g \mu_{gi}))$. We retain DESeq2’s initialization, line search, trend fit, MAP estimate, and outlier rule. Dispersion is often the most expensive stage, so cuDESeq2 provides three ways to run its gradient-ascent loop: eager PyTorch, CUDA-graph replay, and a fused Triton kernel. They make the same optimizer decisions but schedule the work differently.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \dots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P = 1$ or $P > 6$. Triton’s agreement with eager is quantified in Sec. 3.1.

CUDA-graph chunked replay. Eager mode launches several small kernels for each gradient step. The arrays are only 1–10 MB, so launch latency can dominate the calculation. CUDA graphs reduce that overhead by replaying a captured sequence of operations. Two details require special handling.

First, genes converge at different times. Capturing the full loop would force every gene to run for the maximum number of iterations. Instead, we capture a $K=10$ -iteration chunk

and check convergence between replays. State is kept in persistent buffers. Ten iterations balanced replay overhead against the extra work done by genes that converge within a chunk.

95 Second, PyTorch’s `slogdet` and `solve` synchronize with the host and cannot be captured. The Cox–Reid term needs both $\log \det(b)$ and $\text{tr}(b^{-1} db)$, where $b = X^\top W X$. We compute them with an unrolled, no-pivot LU routine. For $P = 2 \dots 6$, it agrees with the standard `linalg` path to about 10^{-15} .

We warm the chunk three times on a side stream before capture. Captured graphs are
100 cached by tensor shape, chunk length, prior use, data type, and device. The prior variance is stored in a scalar buffer, so one graph can serve datasets with different values. Graph mode calls the same arithmetic routine as eager mode and is therefore bit-identical to it.

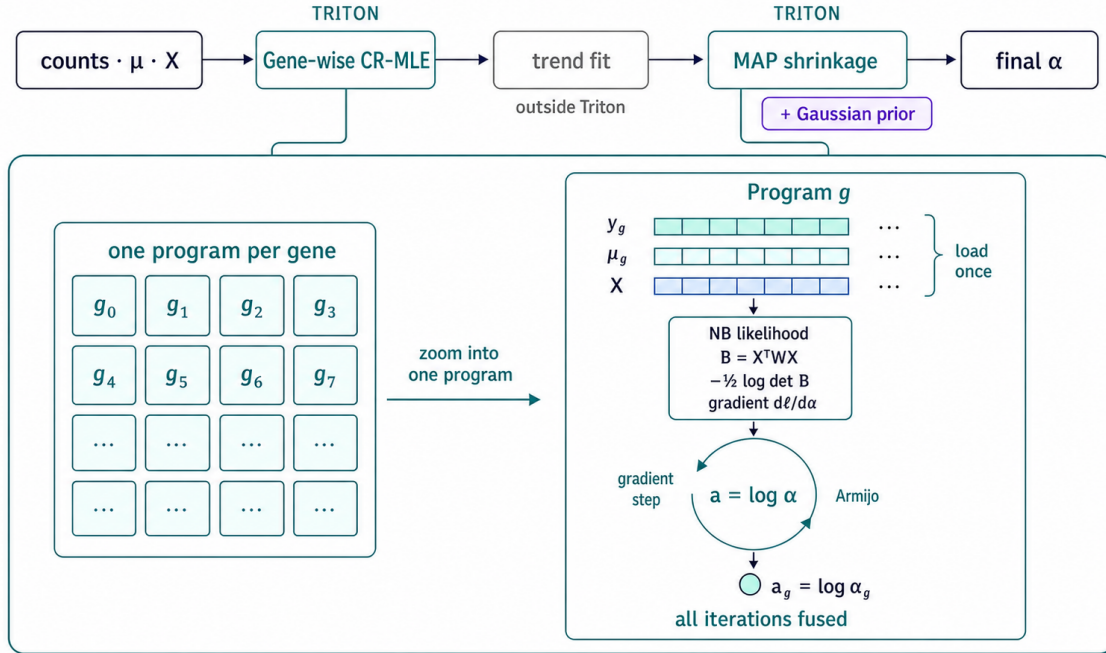


Figure 2: Fused dispersion estimation in Triton. One program handles one gene. It loads the counts, fitted means, and design matrix once, then keeps the gradient-ascent loop inside the kernel. The kernel is used for both gene-wise fits; the shared trend fit remains outside Triton.

Triton full-loop fusion. Triton mode launches one program per gene (Fig. 2). The program loads the gene’s counts, fitted means, and design columns once. It then keeps the

likelihood, gradient, and Armijo line search in registers. Each gene exits when it converges.

Small design matrices are solved directly. We use a closed-form solution for $P = 2$ and unrolled no-pivot LU for $P = 3, \dots, 6$. The kernel uses fp64 arithmetic. Wider designs fall back to eager mode; the low-level launcher rejects them rather than silently truncating the model.

After the kernel returns, non-converged and high-dispersion genes receive the same fallback treatment as in eager mode. Triton changes the floating-point reduction order and uses its own implementation of `digamma`. It is therefore not bit-identical to eager mode. We measure the resulting differences in Sec. 3.1. Full pseudocode is given in Supplementary Algorithms S1–S2.

2.3 GLM acceleration

The negative-binomial IRLS solve is batched across genes. Each iteration uses one batched factorization. Genes that diverge or reach DESeq2’s iteration cap use the reference fallback; only those rows move to the host. Cook’s distances, count replacement, and refitting stay on the GPU. Wald standard errors retain the $\mu \geq 0.5$ floor used by DESeq2, and LRTs use the full-reduced deviance difference.

2.4 LFC shrinkage acceleration

apeGLM (Zhu et al., 2019) also optimizes one model per gene. cuDESeq2 batches its L-BFGS state, line search, and convergence flags. It keeps the reference initialization and stopping rules, which matter for extreme, low-count genes with competing posterior modes. As in `lfcShrink`, only the coefficient and its standard error are replaced. P-values still come from the Wald fit.

Several changes reduce repeated work. The loss and gradient share one gene-sample linear predictor. Terms that do not change during optimization are computed once. Existing size factors and Wald results are reused. On CUDA, PyTorch compilation fuses the elementwise

130 work around the matrix products. CPU execution keeps the eager path.

2.5 Validation and benchmark design

We first tested numerical agreement on five simulated fixtures generated with R DESeq2 1.52.0. Three use a single-factor design with $P = 2$: 12 samples and 200 genes, 30 and 500, and 60 and 2,000. A fourth has 60 samples, 1,500 genes, and a two-factor design with
135 $P = 4$. The fifth has 60 samples, 1,000 genes, and a continuous covariate. The respective random seeds are 0, 0, 1, 2, and 3.

These fixtures test intermediate values throughout the pipeline. They include size factors; gene-wise, trend, MAP, and final dispersions; GLM coefficients and fitted means; Cook’s distances; Wald standard errors; LRT statistics; independent filtering; and apeGLM outputs.
140 Single-factor fixtures also test the local and mean dispersion trends.

We then compared full results on real data. Positive quantities use relative error; values that cross zero use absolute error. Significant sets use Jaccard similarity at $\text{padj} < 0.05$, and shrunk LFCs use Pearson correlation. An empty union has Jaccard similarity one. Eager, graph, and Triton modes were tested separately against the same reference and acceptance
145 thresholds.

Performance was measured on one NVIDIA A100-SXM4-40GB with PyTorch 2.7.1 and CUDA 12.6. The host had 12 virtual CPUs. The reference used R DESeq2 1.52.0, apeglm 1.34.0, and `MulticoreParam(12)`. Both implementations ran the ordinary `DESeq()`, `results()`, and `lfcShrink()` workflow.

150 GPU results are medians of five synchronized runs after one warm-up. The two long GTEx CPU endpoints are single cold runs and are reported as endpoint comparisons. OOM tests are single runs in fresh processes.

The real-data benchmark contains six designs from three published datasets. Pasilla contributes one- and two-factor designs with seven samples (Brooks et al., 2011). Airway
155 contributes three design slices with eight samples (Himes et al., 2014). GTEx contributes

a 300-sample contrast between whole blood and skeletal muscle at 54,922 genes (GTEx Consortium, 2020).

The GTEx counts come from the recount2 reprocessing of SRP012682 (Collado-Torres et al., 2017). We selected 150 samples per tissue with R seed 1. Base coverage was divided
160 by each sample’s mean read length; missing or non-positive lengths were set to 100 bases. A separate fixed-seed simulation varies sample count while holding the gene count at 2,000. Nested cohorts drawn from all 9,662 source samples and 54 tissues test larger models.

3 Results

3.1 GPU execution preserves DESeq2 results

165 We compared each stage of the pipeline with R DESeq2 1.52.0 across six real-data designs derived from three datasets (Table 2). Agreement was evaluated separately for normalization, dispersion estimation, GLM fitting, significance calls, and LFC shrinkage.

Table 2: Agreement with R DESeq2 1.52.0 across six real-data designs. Each design was evaluated independently using a metric appropriate to the indicated pipeline stage.

Stage	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
GLM fit	p95 $ \Delta $ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
LFC shrinkage	Pearson r	0.99999	0.99722

All five stages met their acceptance thresholds. Normalization was effectively identical to R. The worst p95 dispersion error was 0.44%, and the worst p95 raw-LFC error was
170 8.1×10^{-5} . Significant sets had Jaccard similarity of at least 0.99963; shrunken LFCs had Pearson correlation of at least 0.99722.

3.1.1 Parity in standard DE visualizations

The standard diagnostic plots tell the same story (Fig. 3). The dispersion trends, shrunk effects, and significance patterns overlap. Both implementations call 3,993 genes at $\text{padj} < 0.05$.

175 0.05.

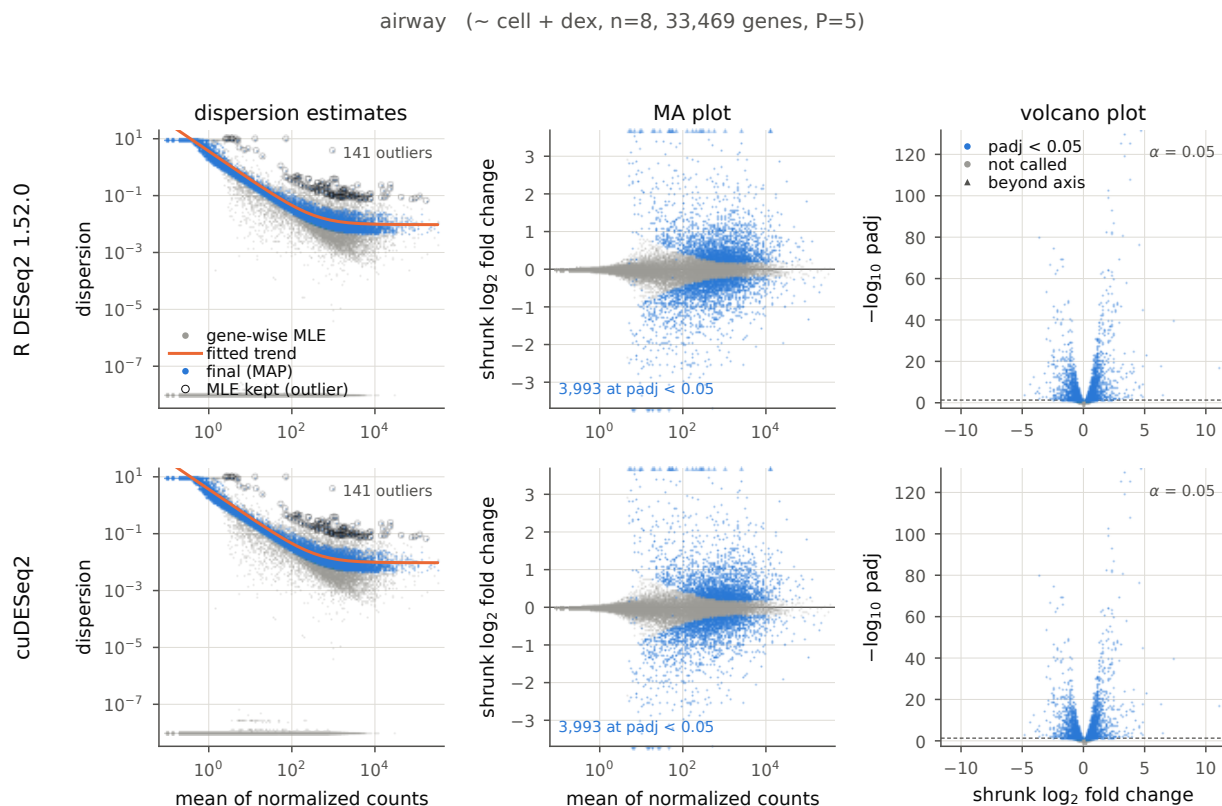


Figure 3: Matched diagnostic plots for **airway** ($\sim \text{cell} + \text{dex}$, $n=8$, $P=5$). R DESeq2 is shown above and cuDESeq2 below. Columns show dispersion estimates, the apeGLM-shrunk MA plot, and the volcano plot. Both rows use the same code and axes.

Ranked genes also agree (Fig. 4). At the prespecified $\alpha=0.05$, four called sets are identical. The remaining two have Jaccard indices of 0.99963 and 0.99997.

3.2 GPU acceleration reduces end-to-end wall time

Every GPU mode was faster than 12-worker R DESeq2 on all six designs (Table 3). The gain depended on the design. Graph and Triton modes change only dispersion execution, so

180

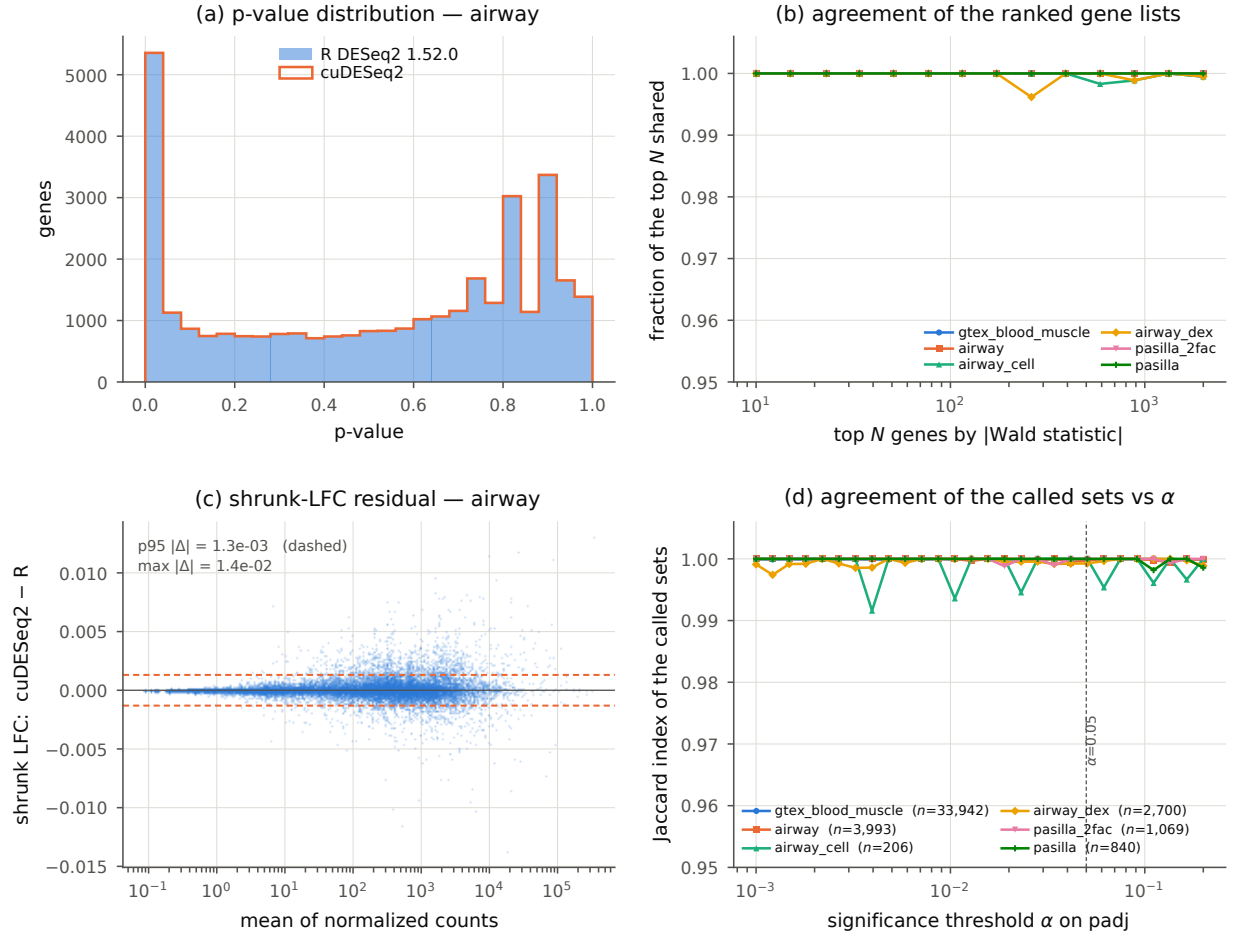


Figure 4: Agreement across the six real-data designs. (a) Overlaid p-value distributions for **airway**. (b) Overlap among the top N genes ranked by absolute Wald statistic. (c) Difference in shrunken LFC versus expression. (d) Jaccard index of significant sets across adjusted-p thresholds.

the stage timings show where their extra speed comes from (Table 4).

Table 3: End-to-end wall time on real data. R uses 12 `BiocParallel` workers. GPU values are medians of five complete runs. All timings include setup, inference, LFC shrinkage, and outlier refitting. The fastest time in each row is bold.

Dataset	P	n	R 12w (s)	eager (s)	graph (s)	Triton (s)	best GPU vs. R
pasilla	2	7	7.336	0.972	0.588	0.525	14.0×
pasilla_2fac	3	7	7.371	1.541	0.932	0.848	8.7×
airway_dex	2	8	11.856	1.332	0.858	0.790	15.0×
airway_cell	4	8	10.421	3.596	3.003	2.870	3.6×
airway	5	8	12.099	2.479	1.552	1.401	8.6×
gtex	2	300	63.445	5.284	4.910	1.807	35.1×

The fastest GPU mode was 3.6–35.1× faster than the CPU baseline. Triton had the lowest end-to-end time and the fastest dispersion stage on every design. For example, it reduced dispersion time from 777 to 87 ms on `pasilla_2fac`, an 8.9× gain. On `airway`, the
185 reduction was 1660 to 585 ms, or 2.8×. Graph mode was slower than Triton but remained bit-identical to eager mode.

3.3 Fused dispersion remains efficient as sample count grows

We next fixed the gene count at 2,000 and varied the sample count from 4 to 2,000 (Table 5). Triton was fastest throughout. Times were not monotonic because the simulated inputs
190 required different numbers of optimizer iterations. Graph replay helped most when launch overhead dominated.

3.4 Real GTEx cohorts reveal scaling and memory limits

The synthetic sweep covers only dispersion. To test the complete workflow, we built nested cohorts from the 9,662-sample recount2 GTEx study. Every cohort contains 54,922 genes.
195 One series keeps $P = 2$ and adds blood and muscle samples. A second series adds tissues, increasing both sample count and design width.

At $P = 2$, increasing the cohort from 300 to 912 samples raised GPU time from 1.753 to 5.430 seconds. Peak allocation rose from 1.98 to 5.99 GiB. The largest fused model used

Table 4: Pipeline-stage wall time (ms) for the three GPU execution modes. Bold values identify the fastest mode for each stage and total. GLM fitting includes the initial Wald fit and count-outlier replacement/refitting. The *total* is the sum of stage medians, not a separate end-to-end measurement.

Dataset	Stage	eager	graph	Triton
pasilla	dispersion	463	78	21
	GLM fit	42	42	41
	significance	40	40	40
	LFC shrinkage	420	422	421
	<i>total</i>	966	583	524
pasilla_2fac	dispersion	777	174	87
	GLM fit	354	354	352
	significance	50	50	49
	LFC shrinkage	347	346	347
	<i>total</i>	1528	925	837
airway_dex	dispersion	581	141	56
	GLM fit	58	59	57
	significance	82	109	82
	LFC shrinkage	573	578	576
	<i>total</i>	1296	888	773
airway_cell	dispersion	801	198	62
	GLM fit	39	38	39
	significance	78	78	75
	LFC shrinkage	2675	2667	2674
	<i>total</i>	3593	2982	2852
airway	dispersion	1660	723	585
	GLM fit	62	62	64
	significance	74	74	73
	LFC shrinkage	673	685	663
	<i>total</i>	2470	1545	1386
gtex	dispersion	3574	3612	217
	GLM fit	849	491	432
	significance	85	73	95
	LFC shrinkage	570	569	929
	<i>total</i>	5081	4749	1678

normalization omitted (≤ 4 ms cuDESeq2 on every set). All GPU values come from the same A100 session.

Table 5: Sample-count scaling of the full dispersion stage (ms) at 2,000 genes. Values are CUDA-synchronized medians of seven measurements after three warm-ups on one A100; the simulator seed is 7919*n*.

Samples	eager	graph	Triton
4	1075	636	498
6	627	161	32
30	187	68	28
60	238	61	30
200	239	113	34
1000	337	282	23
2000	474	402	90

Table 6: Full-workflow real-GTEx scaling on one A100 at 54,922 genes. Direct wall time is the median of five CUDA-synchronized observations after one warm-up; peak allocation is the maximum over five staged repetitions. P=2–6 uses the fused Triton dispersion path.

Cohort	Samples	P	Direct (s)	Peak GiB
paper-equivalent	300	2	1.753	1.98
2 tissues, balanced	600	2	2.740	3.95
2 tissues, source max.	912	2	5.430	5.99
3 tissues $\times$ 300	900	3	6.005	5.92
4 tissues $\times$ 300	1,200	4	8.990	8.38
5 tissues $\times$ 300	1,500	5	9.898	11.09
6 tissues $\times$ 300	1,800	6	14.323	14.77
6 tissues, all samples	2,451	6	22.936	20.10

all 2,451 samples from six tissues. It completed in a 22.936-s median and used 20.10 GiB at
 200 peak.

We also ran the 912- and 2,451-sample endpoints with 12-worker R DESeq2 (Table 7). The GPU was $39.4\times$ and $28.0\times$ faster, respectively. Each CPU value is one cold run, so these ratios should be read as endpoint comparisons rather than estimates of a timing distribution.

Table 7: Large-GTEx endpoints against 12-worker R. R values are single cold runs; GPU values are medians of five warm runs. Both include dataset construction and the complete workflow. The faster time is bold.

Samples	P	GPU (s)	R 12w (s)	GPU vs. R 12w
912	2	5.430	213.896	$39.4\times$
2,451	6	22.936	642.552	$28.0\times$

The large endpoints also passed all five parity checks. At $P = 6$, the p95 dispersion error
 205 was 0.00212. The significant-set Jaccard index was 0.99896, and shrunken-LFC correlation was 0.999994. Dispersion was compared immediately after estimation and before outlier refitting.

Finally, we increased design width beyond Triton’s $P \leq 6$ limit. These models use eager dispersion (Table 8). A $P = 9$ model with 3,478 samples completed at 37.09 GiB peak
 210 allocation. The next model, $P = 10$ with 3,784 samples, failed during dispersion. This is an observed boundary for these nested cohorts, not a general limit on sample count.

Table 8: Observed A100 feasibility boundary for nested all-sample GTEx tissue models. Each row is one cold workflow in a fresh process. $P > 6$ uses eager dispersion fallback; pass-row wall times are not treated as timing estimates.

Tissues	Samples	P	Peak GiB	Status	Failed stage
8	3,147	8	30.98	pass	—
9	3,478	9	37.09	pass	—
10	3,784	10	27.88	OOM	dispersion
12	4,338	12	35.51	OOM	dispersion

4 Discussion

cuDESeq2 shows that the DESeq2 workflow can use GPU parallelism without switching to a different test. It matched R at each pipeline stage and was $3.6\text{--}35.1\times$ faster end to end.

215 Checking intermediate values was important. Agreement in the final gene list alone could hide errors in dispersion, standard errors, or multiple-testing correction.

The useful unit of parallelism is the gene. Basic batching sends thousands of independent fits to the GPU at once. Dispersion estimation needs more than batching because it consists of small operations inside variable-length loops. CUDA graphs reduce launch overhead while
220 preserving eager arithmetic. Triton goes further by fusing each gene’s loop into one kernel. The best choice depends on the size of the design and the number of samples.

Numerical details matter when matching a reference implementation. The model equations do not specify every optimizer decision or floating-point operation. Triton, for example, changes reduction order and supplies its own `digamma`. The largest eager–Triton dispersion
225 difference was 0.2604 on GTEx, compared with at most 3.8×10^{-7} on the five smaller designs. These differences did not materially change downstream results in our tests, but Triton is not bit-identical to eager mode.

GTEx shows both the reach and the present memory limit. The fused path handled 2,451 samples and six tissues using about half of a 40-GB A100. Wider eager models reached the
230 boundary between $P = 9$ and $P = 10$. Both sample count and design width increase memory use, so this boundary is specific to the tested cohorts. Chunking or a wider fused kernel could move it. Either change would need a new parity analysis.

The strongest use case is repeated analysis: perturbation screens, resampling, many contrasts, or evaluation of generated expression profiles. For one small experiment, access
235 to a GPU and added software complexity may outweigh the time saved. cuDESeq2 is most useful when compatibility with existing DESeq2 analyses matters and the same workflow must run many times.

This study has several limits. All timings come from one A100 and one 12-vCPU host.

Five of the six primary designs have only seven or eight samples, and the large-scale study
240 uses one GTEx source. Interactions, observation weights, `ashr`, normal shrinkage, and the
`glmGamPoi` dispersion backend were not validated. Triton supports $P = 2, \dots, 6$; wider
designs use eager dispersion. The large-GTEx CPU endpoints and OOM probes are single
observations.

Within this scope, gene-batched GPU execution preserves DESeq2-style results while
245 cutting analysis time. It offers a practical alternative to replacing a count model with a
cheaper surrogate in high-throughput settings.

Funding

This work was funded by Synthesize Bio.

Conflict of Interest

250 All authors are affiliated with Synthesize Bio, which developed and maintains the software
described here and released it under the MIT license. The authors declare no other competing
interests.

Data and software availability

The `pasilla` benchmark uses the *Drosophila melanogaster* RNA-seq experiment dis-
255 tributed in the Bioconductor `pasilla` 1.40.0 source package (Brooks et al., 2011).

The `airway` benchmark uses the human airway smooth-muscle-cell experiment in the
Bioconductor `airway` 1.32.0 source package (Himes et al., 2014). The GTEx whole-
blood versus skeletal-muscle benchmark uses `recount2` gene-level data for SRA study
SRP012682 (<https://recount-omndata.s3.amazonaws.com/recount2/v2/SRP012682/>
260 `rse_gene.Rdata`) (GTEx Consortium, 2020; Collado-Torres et al., 2017).

Code and the reproducibility guide are available at https://github.com/synthesizebio/gpu_deseq under the MIT license. The repository records input checksums, GTEx sample selections, raw timings, parity metrics, memory use, software versions, and container digests. The 1.37-GB recount2 object is downloaded from the public URL above rather than copied into the repository.

References

A. K. Adduri, D. Gautam, B. Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *bioRxiv*, 2025. doi: 10.1101/2025.06.26.661135.

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma–poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory map between *Drosophila* and mammals. *Genome Research*, 21(2):193–202, 2011. doi: 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammerers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

- 285 GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.
- Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014.
- 290 doi: 10.1371/journal.pone.0099625.
- Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.
- 295 G. Koytiger, A. M. Walsh, V. Marar, et al. Generative genomics accurately predicts future experimental results. *bioRxiv*, 2025. doi: 10.1101/2025.09.08.674753.
- Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.
- 300 Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9):btad547, 2023. doi: 10.1093/bioinformatics/btad547.
- NVIDIA Corporation. CUDA C++ programming guide: CUDA graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024. Accessed July 2026.
- 305 Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a

Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

310 Christina V. Theodoris, Ling Xiao, Anant Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.
315

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for
320 sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary benchmark details

The benchmark host was a KVM virtual machine with a 2.20-GHz Intel Xeon CPU, 6 physical cores, 12 logical CPUs, 83 GiB RAM, and Linux 5.10. GPU runs used one NVIDIA A100-SXM4-40GB. The software stack was PyTorch 2.7.1, CUDA 12.6, and NVIDIA driver 580.126.20. The R stack was R 4.6.0, DESeq2 1.52.0, and apeglm 1.34.0.

OMP, OpenBLAS, and MKL were pinned to one thread per R worker. Main-suite and real-GTEX GPU values are medians of five runs after one warm-up. The two large GTEX CPU endpoints and four OOM probes are single runs. The synthetic sample sweep uses seven measurements after three warm-ups.

Supplementary algorithms

Supplementary Algorithm S1 summarizes the fused Triton fit. Supplementary Algorithm S2 gives the full kernel, including its Armijo line search, step-size update, and Cox–Reid calculations.

Supplementary Algorithm S1 FUSED TRITON DISPERSION FIT (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.

Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

- 1: **Launch** one Triton program for every gene g in parallel.
 - 2: Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
 - 3: Initialize $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
 - 4: **repeat** inside the same fused program
 - 5: Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
 - 6: If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
 - 7: Compute the gradient $q \leftarrow \partial \ell / \partial a$.
 - 8: Propose the gradient-ascent step $\tilde{a} \leftarrow a + \kappa q$.
 - 9: Accept or reject $\tilde{a}$ using the Armijo condition; adapt κ .
 - 10: **until** converged or *maxit* is reached.
 - 11: Store the final a_g and iteration count c_g once in global memory.
-

Supplementary Algorithm S2 DETAILED FUSED TRITON DISPERSION KERNEL (CR-MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$, $P \in \{2, \dots, 6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^G$; compile-time $h \equiv \text{HASPRIOR}$; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0 .

Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

```

1: parallel for Triton program  $g \in \{0, \dots, G-1\}$  do (one program per gene)
2:   Load  $\mathbf{y}_g$ ,  $\boldsymbol{\mu}_g$ , and  $\mathbf{X}$  once into program-local state.
3:   Set  $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ ,  $\kappa \leftarrow \kappa_0$ ,  $n_{\text{acc}} \leftarrow 0$ , and  $c \leftarrow 0$ .
4:   Set  $(\ell, q) \leftarrow \text{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
5:   while  $c < \text{maxit}$  and not converged do
6:      $c \leftarrow c + 1$  and propose  $\tilde{a} \leftarrow a + \kappa q$ .
7:     If  $q \neq 0$  and  $\tilde{a} \notin [-30, 10]$ , shorten  $\kappa$  so that  $\tilde{a}$  lies on the violated boundary.
8:     Set  $(\tilde{\ell}, -) \leftarrow \text{ObjectiveGradient}(\tilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
9:     Set  $r \leftarrow [\tilde{\ell} \geq \ell + \varepsilon \kappa q^2]$ . (Armijo acceptance flag)
10:    Select  $a' \leftarrow r ? \tilde{a} : a$ ,  $\ell' \leftarrow r ? \tilde{\ell} : \ell$ , and  $\Delta \leftarrow \ell' - \ell$ .
11:    Set  $(-, q') \leftarrow \text{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$  and  $q \leftarrow r ? q' : q$ .
12:    Set  $n'_{\text{acc}} \leftarrow n_{\text{acc}} + r$  and  $\kappa_{\text{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$ .
13:    If  $r$  and  $n'_{\text{acc}} \bmod 5 = 0$ , set  $\kappa_{\text{acc}} \leftarrow \kappa_{\text{acc}}/2$ .
14:    Set  $\kappa \leftarrow r ? \kappa_{\text{acc}} : \kappa/2$ ,  $n_{\text{acc}} \leftarrow n'_{\text{acc}}$ ,  $a \leftarrow a'$ , and  $\ell \leftarrow \ell'$ .
15:    Converge if  $r \wedge (\Delta < 10^{-6} \vee a < \log(\text{minDisp}/10))$ .
16:  end while
17:  Store  $a_g \leftarrow a$  and  $c_g \leftarrow c$  in global memory.
18: end parallel for
19: function ObjectiveGradient( $a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2$ )
20:   Set  $\alpha \leftarrow e^a$ ,  $\mathbf{W} \leftarrow \text{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$ , and  $\mathbf{B} \leftarrow \mathbf{X}^\top \mathbf{W} \mathbf{X}$ .
21:   Compute  $\ell \leftarrow \ell_{\text{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2} \log \det(\mathbf{B})$ .
22:   Compute  $q \leftarrow \partial \ell / \partial a$ , including  $\text{tr}(\mathbf{B}^{-1} \partial \mathbf{B} / \partial \alpha)$  via an unrolled  $2 \times 2$  formula ( $P=2$ ) or  $P \times P$  LU solve.
23:   if  $h = 1$  then  $\ell \leftarrow \ell - (a - m)^2 / (2\sigma^2)$  and  $q \leftarrow q - (a - m) / \sigma^2$ . (MAP only)
24:   return  $(\ell, q)$ .
25: end function

```
